

# AI Accelerator Design – Assignment 3

## Tensor Flattening and Reconstruction

### 1. Objective

The objective of this assignment is to map a tensor in BCHW/NCHW format from  $B \times C \times H \times W$  to  $B \times (CHW)$  and reconstruct it back to its original BCHW representation. The implementation uses explicit loops and index arithmetic rather than library shape-conversion functions such as `reshape()`, `flatten()`, `ravel()`, or `view()`.

### 2. Flattening Algorithm

For an element  $I[b, c, h, w]$ , the corresponding position  $k$  in the flattened CHW dimension is calculated as:

$$k = c \times H \times W + h \times W + w$$

The value is then copied using  $F[b, k] = I[b, c, h, w]$ . The algorithm iterates over batch, channel, height, and width indices.

#### Flattening Pseudocode

```
for b = 0 to B-1
  for c = 0 to C-1
    for h = 0 to H-1
      for w = 0 to W-1
        k = c*H*W + h*W + w
        F[b, k] = I[b, c, h, w]
```

### 3. Reconstruction Algorithm

For reconstruction, the flattened index  $k$  is converted back into channel, height, and width indices. First, the channel is obtained using integer division. The remaining index is then used to recover height and width:

$$\begin{aligned} c &= k // (H \times W) \\ r &= k \% (H \times W) \\ h &= r // W \\ w &= r \% W \end{aligned}$$

Finally, the value is copied using  $I_{\text{hat}}[b, c, h, w] = F[b, k]$ .

#### Reconstruction Pseudocode

```
for b = 0 to B-1
  for k = 0 to C*H*W-1
    c = k // (H*W)
    r = k \% (H*W)
    h = r // W
    w = r \% W
    I_hat[b, c, h, w] = F[b, k]
```

### 4. Manual Indexing Example

Consider  $B = 1$ ,  $C = 2$ ,  $H = 2$ , and  $W = 3$ . For one batch, the two channels contain 6 elements each. The flattened ordering is:

$I[0,0,0,0]$ ,  $I[0,0,0,1]$ ,  $I[0,0,0,2]$ ,  $I[0,0,1,0]$ ,  $I[0,0,1,1]$ ,  $I[0,0,1,2]$ ,  $I[0,1,0,0]$ ,  $I[0,1,0,1]$ ,  $I[0,1,0,2]$ ,  $I[0,1,1,0]$ ,  $I[0,1,1,1]$ ,  $I[0,1,1,2]$

For example, for  $(c, h, w) = (1, 0, 2)$ :  $k = 1 \times 2 \times 3 + 0 \times 3 + 2 = 8$ . Therefore,  **$I[0,1,0,2]$  maps to  $F[0,8]$** . During reconstruction,  $k = 8$  gives  $c = 8 // 6 = 1$ ,  $r = 8 \% 6 = 2$ ,  $h = 2 // 3 = 0$ , and  $w = 2 \% 3 = 2$ , recovering the original position.

### 5. Experimental Setup

The experiments used synthetic tensors with  $B = 2$ ,  $H = 8$ , and  $W = 8$ . The channel count  $C$  was varied to test both small and large feature-map sizes. The tested channel counts were 1, 3, 8, 16, 32, 64, 128, 256, and 500.

| B | C   | H | W | E <sub>max</sub> | MAE |
|---|-----|---|---|------------------|-----|
| 2 | 1   | 8 | 8 | 0.0              | 0.0 |
| 2 | 3   | 8 | 8 | 0.0              | 0.0 |
| 2 | 8   | 8 | 8 | 0.0              | 0.0 |
| 2 | 16  | 8 | 8 | 0.0              | 0.0 |
| 2 | 32  | 8 | 8 | 0.0              | 0.0 |
| 2 | 64  | 8 | 8 | 0.0              | 0.0 |
| 2 | 128 | 8 | 8 | 0.0              | 0.0 |
| 2 | 256 | 8 | 8 | 0.0              | 0.0 |
| 2 | 500 | 8 | 8 | 0.0              | 0.0 |

## 6. Results and Discussion

For all nine tensor configurations, the maximum absolute reconstruction error (E<sub>max</sub>) and mean absolute error (MAE) were exactly 0.0. This shows that every tensor element was mapped to the correct flattened location and subsequently restored to its original BCHW position.

The flattened tensor sizes also increased as expected. For example, C = 1 produced a B × 64 tensor, while C = 500 produced a B × 32000 tensor. Despite this increase in channel count, the explicit indexing algorithm continued to reconstruct the tensors without error.

The zero reconstruction errors are consistent with the fact that the conversion operations only copy tensor values and do not modify them.

## 7. Conclusion

The BCHW-to-B×(CHW) flattening and reverse reconstruction algorithms were successfully implemented using explicit indexing and arithmetic. The experiments across channel counts from 1 to 500 produced zero maximum absolute error and zero mean absolute error, confirming correct address mapping and reconstruction.

## 8. GitHub Repository

[https://github.com/Satvik-ai/Tensor\\_Flattening\\_and\\_Reconstruction](https://github.com/Satvik-ai/Tensor_Flattening_and_Reconstruction)